Mohammed Ali Al-Saiaf - 310320

Authentication, Access Control, and WDAC

1. User Authentication: Passwords

HTWG has ca. 5,000 user accounts that are protected by passwords chosen by users. Each password needs to be at least 14 characters long. Assuming that users prefer simple passwords that are exactly 14 characters long, how many different passwords are possible when users only use lower case characters a...z for their passwords?

- Number of characters: 26 (lowercase a-z)
- Password length: 14
- Total number of passwords = 26^14

1.1. An attacker might perform an online attack on the passwords of HTWG's users. Assume that the probability of all passwords is equal. Assume further that the attacker can try 30,000 different passwords per second. How many days in total does the attacker have to guess passwords until the attacker guesses the correct password for a specific account?

$$\begin{aligned}\text{Total number of passwords} &= 26^{14} \\ \text{Total number of guesses per second} &= 30,000 \\ \text{Total number of guesses every day} &= 2,592,000,000\end{aligned}$$

Best case scenario the attacker can guess the password instantly, however, because it is unlikely for the attacker to guess the password instantly, I will calculate the average case scenario. Assuming it takes the attacker half the numbers of guesses to guess the password:

$$
\begin{aligned}
\text{Average number of guesses} &= \frac{26^{14}}{2} \\
\text{Average number of days} &= \frac{\text{Average number of guesses}}{\text{Total number of guesses every day}} \\
&= \frac{26^{14}}{2 \cdot 2,592,000,000} \\
&= \frac{26^{14}}{5,184,000,000} \\
&\approx 1.244 \times 10^{10} \text{ days} \\
&\approx 34,000,000 \text{ years}
\end{aligned}
$$

Or assuming it takes all possible tries to guess the password (worst case scenario):

$$
\begin{aligned}
\text{Total number of days} &= \frac{26^{14}}{2,592,000,000} \\
&= \frac{26^{14}}{2,592,000,000} \\
&\approx 2.488 \times 10^{10} \text{ days} \\
&\approx 68,000,000 \text{ years}
\end{aligned}
$$

1.2. Assume that the probability of passwords is not equal. Assume that 90% of the users choose their passwords from a set of 1,000 popular passwords. Assume further that the attacker can try 10,000 different passwords per second. How many seconds on average does the attacker have to guess passwords until the attacker guesses the correct password for at least 90 accounts?

For 5000 users, if 90% of them (4500) choose their passwords from a set of 1000 popular passwords and the attacker can try 10,000 different passwords per second, we can calculate the average time by taking half of the number of popular passwords:

$$
\frac{1000}{2} = 500
$$

Then we can calculate attempts needed to guess the password for 90 accounts:

$$
500 \times 90 = 45,000 \text{ passwords attempts}
$$

So the time needed to guess the password for 90 accounts is:

$$
\begin{aligned}
\text{Total number of seconds} &= \frac{45,000}{10,000} \\
&= 4.5 \text{ seconds}
\end{aligned}
$$

Assume that a) the attacker can only perform an online attack against one specific account (you do not know in advance which one), and b) that 90% of the users choose their passwords from a

set of 1,000 popular passwords. Assume further that you want the attacker to be unsuccessful with a probability of 95%. After how many tries do you need to shut down the authentication mechanism?

if 90% of users use the top 1000 most popular passwords then we need to check only for them because we assume the worst case scenario, that the attacker will try all 1000 passwords.
So let

$$P(\text{success}) = 0.95$$

and also

$$P(\text{success}) = P(A) \cdot \frac{N}{1000} = 0.95 \times \frac{N}{1000}$$

so for the attacker to be unsuccessful with a probability of 95%, we calculate the probability of success of 5%:

$$0.9 \times \frac{N}{1000} \leq 0.05 \Rightarrow N \leq \frac{0.05 \times 1000}{0.9} = 55.5\ldots$$

and rounding down gives us:

$$\boxed{N \leq 55 \text{ attempts}}$$

---

2. Access Control

Restricting access by people/accounts/programs ("subjects") to resources ("objects") is a method to preserve confidentiality, integrity, and availability of objects. There are different approaches how you can define who can perform which actions on what.

2.1. Pick three apps on your smartphone. What is the purpose of each app and what permissions are associated with these apps? Is the association of permissions with apps an access control list or a capability list? Why?

App Name	Purpose	Permissions	ACL or CL	Why?
WhatsApp	Messaging	Camera, Microphone, Contacts, Storage	CL	OS gives permissions to the app directly
Tiktok	Social Media	Camera, Microphone, Location, Contacts	CL	App proves its permissions at runtime
Instagram	Social Media	Camera, Microphone, Storage, Contacts	CL	App presents its access rights, not managed per device

Access Control List vs Capability List:

- An Access Control List typically assigns permissions to specific objects (like files, folders, or resources) and determines who or what can access those objects. It defines what users or apps can do with those objects
- A Capability List, on the other hand, assigns permissions to the subject, specifying what actions or resources the app is capable of interacting with (e.g., camera, location, contacts)

2.2. Explain how role-based access control works. Use Moodle as an example for an application that uses role-based access control.

Role-based access control means that users don't get permissions directly, but instead they are assigned roles and each role has a set of permissions.
This makes it easier to manage many users, because you just change the role once instead of adjusting every user manually.
In Moodle, for example, there are different roles like *student*, *teacher*, and *admin*. Each role has different permissions, like students can only view course materials while teachers can create and edit course content.
When a user is assigned a role, they automatically inherit the permissions associated with that role.

2.3. It is a recommended practice to use separate user accounts for everyday tasks and for system-administrative tasks, even if

the person performing these tasks is the same. Why is this a good practice? Or is this just unnecessary bureaucracy? Explain.

Reason 1: Reducing risk if something goes wrong

If I'm logged in with an admin account all the time, every program I run also has admin rights.
So if malware or a bad website manages to attack my system, it automatically gets full control.
But if I'm using a normal account, even if something bad happens, it's much harder for the attacker to do real damage (like installing ransomware or deleting important system files).

Reason 2: Avoiding mistakes

As a normal user, I simply don't have permission to accidentally change important system settings or delete critical files.
When I really need admin rights, I can log in separately and be more careful because I know I'm doing something sensitive.

---

3. Application Whitelisting

3.1. Explain how application whitelisting works with WDAC (Windows Defender Application Control)?

Policies are created that define trusted apps.
The policy can say: only apps signed by a trusted certificate, or only apps from specific paths, or only specific files are allowed.
When a user or even malware tries to run a program *WDAC* checks the policy. If the app is not on the allowed list, Windows blocks it before it can even start.
The checks happen very early when the program loads before it runs any code.

WDAC works by using things like:

- File hashes (specific fingerprints of allowed files),
- Code signing certificates (apps must be signed by trusted developers)
- Specific file paths or folder rules.

The goal is to prevent unknown or untrusted software from ever running, even if a user downloads something by accident.

3.2. Does WDAC restrict users/processes with respect to executing code? How?

Yes it restricts users and processes by checking if the code they want to run is on the whitelist. If it’s not, Windows blocks it. This means that even if a user tries to run a program that’s not allowed or if malware tries to execute code it wont be able to run because it doesn’t meet the whitelist criteria.

3.3. Does WDAC restrict users/processes with respect to reading and writing files? How?

It doesn't stop users or programs from accessing files. Instead, it just controls which apps can be executed. So even if someone can read or write files, they can't run any software that isn't trusted. The operating system takes care of file access permissions separately and that's not included in its policy.

3.4. What are the different file rules hat you can use with WDAC?

Type of rule	What it does
Publisher rule	Allows all files that are signed by a trusted publisher (e.g., Microsoft, Adobe) very flexible if you trust the company
File path rule	Allows files from specific folders or locations (e.g., only allow programs from `C:\Program Files`), easy to set up but risky if the folder can be modified by users
File hash rule	Allows only files that exactly match a certain cryptographic fingerprint (hash). Super strict even small file changes break the match
Package rules (for MSIX packages)	Allows modern Windows apps (UWP apps) based on their package identity. Useful for Store apps or enterprise-deployed apps

3.5. What is the purpose of the audit mode in WDAC?

Audit mode lets you check *WDAC* policies without stopping any apps. It records what would have been blocked if the rules were active. This way, you can see which apps might be impacted and adjust the policy before fully enforcing it. It's like a practice run to ensure everything goes smoothly without any interruptions.

3.6. How do you debug WDAC policies, i.e., how do you find out that they are effective and where are WDAC events logged?

You need to check the event logs in Windows. Whenever *WDAC* permits or denies an application (or would deny it in audit mode) it logs an entry in the Windows Event Viewer. When you access this log you'll see information about what occurred, like whether an app was allowed or denied, which rule caused the decision and the reasoning behind it. If something crucial gets blocked or if unexpected applications attempt to run you can catch it here.
If you're testing a policy in audit mode this is also where you can see which applications would have been blocked allowing you to tweak the *WDAC* policy before putting it into effect.